

PRÁCTICA 4

Elena Abreu Fernández y Tsan-Yu Wu



**UNIVERSIDAD
DE GRANADA**

1. Cambios respecto a la práctica anterior

Para la realización de esta práctica es necesario utilizar los índices creados con Lucene en la entrega anterior. A la hora de crear las consultas necesarias, nos dimos cuenta de algunas carencias y mejoras que podíamos aplicar a nuestro código de indexación :

- Los atributos relacionados con URL e ID se han modificado para que simplemente se almacenen (StoredField), sin crearlos como campos StringField.
- Al realizar consultas ordenadas y numéricas nos dimos cuenta de que los resultados no eran claros debido a que tratamos con datos incompletos, pues algunos AirBnB no tienen todos sus campos rellenos. Es por ello que hemos reforzado la limpieza de datos como review_scores_rating, bathrooms, bedrooms, number_of_reviews y price asegurándonos de que al indexar los datos se guardan con un valor de cero en caso de estar vacíos en el CSV.
- En relación a lo anterior, también hemos modificado price y reviews_score_rating para que además se añadan al índice como DoubleDocValuesField, lo que nos permitirá ordenar las salidas de las consultas según sus valores.
- Para poder hacer las consultas geográficas correctamente se guarda location como LatLonDocValuesField.

2. Consultas QueryParser

Para realizar esto, creamos la función independienteConsulta y pasamos los IndexSearcher de cada index. Aquí replicamos el ejemplo en el .pdf con dos IndexSearcher.

3. Consultas numéricas

Para crear consultas numéricas, usamos newExactQuery, newRangeQuery, y newSetQuery. Hemos creado newExactQuery para buscar apartamentos que tiene solo exactamente un baño; newRangeQuery para buscar apartamentos que cuestan desde 50 dollars hasta 300 dollars; newSetQuery para buscar apartamentos que tienen 1 o 4 baños. Cuando implementamos, nos dimos cuenta que había problemas de valor en "bathrooms". Lo de antes no funcionaba correctamente, por ejemplo, el número "1.0" se almacenaba como "10".

4. Consultas BooleanQuery

Aquí usamos el index de anfitrión y los campos: host_about, host_response_time, y host_neighbourhood. Buscamos anfitriones que: se describe con la palabra "cool" (a saber: Term("host_about", "cool"), contesta durante un día y tiene el apartamento en "Venice". Nos parece raro que el programa solo devuelva resultados si las consultas en los campos host_response_time y host_neighbourhood tienen la condición BooleanClause.Occur.SHOULD, aunque algunos resultados ya satisfacen los dos otros campos. Es decir, hay resultado que contiene "cool" en el campo host_about y también "Venice" en host_neighbourhood, pero si ponemos BooleanClause.Occur.MUST en su TermQuery, no se encuentra ningún apartamento.

5. Consultas ordenadas

Para este apartado hemos decidido hacer una consulta acerca de apartamentos cerca de la playa y a continuación ordenamos los resultados de distintas formas, según lo que creemos que puede resultar de mayor interés para el usuario : el precio y la opinión del resto de usuarios.

Hemos comenzado ordenando la salida de la consulta de manera ascendente según su precio, es decir, obtendremos más arriba los apartamentos cerca de la playa que son más baratos.

Además decidimos ordenar también los resultados de manera descendente según su puntuación en las reseñas.

6. Consultas geográficas

En lo relacionado a la localización de los apartamentos, hemos realizado tres tipos de consulta: la primera nos mostrará apartamentos en un radio de 2 km, la segunda en un área cuadrada de lado lon + 1km, y, por último, mostramos resultados ordenados por cercanía a la localización requerida.

7. Consultas MultiReader

Para este último apartado, se probó a usar tanto MultiReader como ParallelCompositeReader para conseguir consultas que recorran los índices referidos a Properties y Host simultáneamente, sin embargo, nos encontramos con problemas al usar ambas estructuras. No hemos conseguido obtener resultados en este apartado, tras investigar en distintas páginas cuál podía ser el problema, muchas coincidían en que probablemente nuestros índices tuvieran distinto orden o distinto número de documentos. Revisamos el código de la práctica de indexación pero no conseguimos dar con el problema. Descartamos que se indexen distinto número de documentos pues lo comprobamos con un print y no conseguimos dar con el fallo.

8. Trabajo en grupo

Tsan-Yu ha realizado los apartados 1,2 y 3. Elena los apartados 4,5 y 6.